赛道3 — 智能体开发赛道

基于 n8n 的智能代码审查与质量管理系统

设计说明书

项目名称：基于 n8n 的智能代码审查与质量管理系统

团队：Life Tree

日期：2026 年 4 月

目录

一、问题背景与业务 / 用户痛点分析

1.1 行业背景

1.2 核心痛点

1.3 现有方案的不足

二、解决方案的整体思路与创新点

2.1 核心思路

2.2 整体架构

2.3 创新点

三、技术选型与实现路径

3.1 技术选型理由

3.2 实现路径

3.3 关键节点实现

3.4 AI Prompt 设计

3.5 手动演示前端

四、功能模块描述与交互流程

4.1 功能模块总览

4.2 交互流程图 — 手动审查场景

4.3 交互流程图 — GitLab 自动审查场景

4.4 知识库学习流程（规划）

五、可扩展性与落地应用前景

5.1 跨场景复用

5.2 部署成本与安全性

5.3 与现有系统集成

5.4 后续优化路径

5.5 落地价值

一、问题背景与业务 / 用户痛点分析

1.1 行业背景

在现代软件开发流程中，**代码审查（Code Review）**是保障代码质量、统一团队规范、发现潜在漏洞的核心环节。然而，随着项目规模增长和团队人员扩张，传统的人工代码审查面临着以下瓶颈：

1.2 核心痛点

痛点	具体表现
审查效率低	人工审查耗时，MR/PR 积压严重，拖延发布节奏
审查质量不均衡	初级审查人员容易遗漏深层问题（安全漏洞、边界条件）
规范不统一	不同审查人的关注点差异大，缺乏一致的检查标准
新人学习成本高	新人难以快速理解团队的编码规范和最佳实践
知识难积累	审查过程中发现的问题和改进建议分散在评论中，无法沉淀为可复用的知识库
自动化程度低	现有的 CI/CD 流水线中，代码审查仍然依赖人工介入

1.3 现有方案的不足

- **静态分析工具（ESLint, SonarQube 等）**：只能检测语法和编码规范层面的问题，无法理解业务逻辑、架构意图和安全性
- **AI 辅助工具（GitHub Copilot Review 等）**：通常绑定特定平台（如 GitHub），缺乏与 GitLab 等其他平台的灵活集成能力
- **纯人工审查**：耗时费力、主观性强、难以形成知识库

> **明确说明**：上述工具均无法实现"多平台适配 + 知识库自进化 + 智能建议 + 报告可视化"的一体化闭环。本方案通过 n8n 工作流编排平台，将 AI 大模型能力无缝嵌入到团队现有开发流程中，填补这一空白。

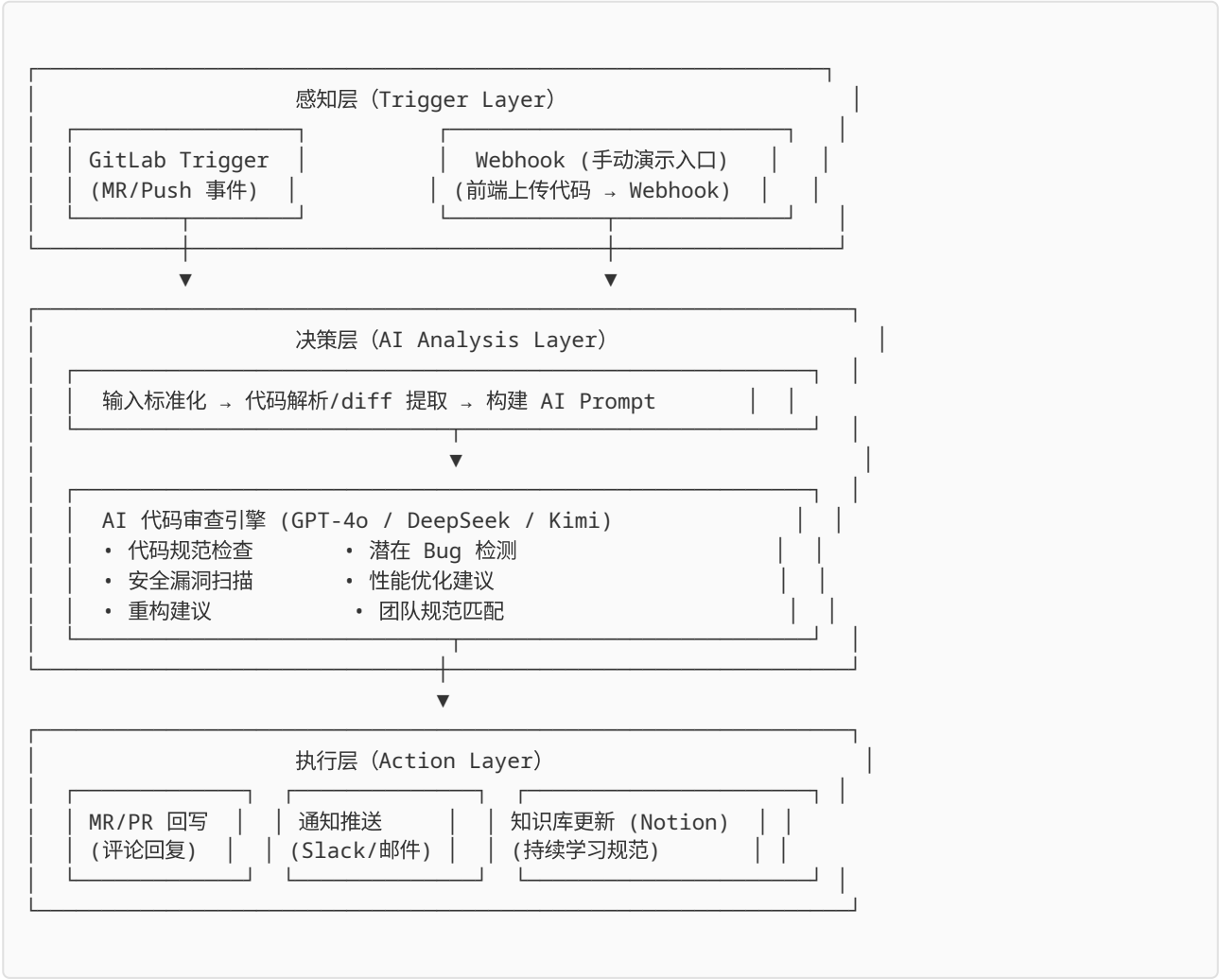
二、解决方案的整体思路与创新点

2.1 核心思路

基于 **n8n** 开源工作流自动化平台，构建一个 **感知-决策-执行闭环** 的智能代码审查系统：

1. **感知（输入）**：自动监听 GitLab/GitHub 的代码提交 / Merge Request 事件；同时支持手动上传代码进行即时审查
2. **决策（AI 分析）**：利用 AI 大模型（GPT-4o / DeepSeek / Kimi）对代码变更进行多维度审查，包括代码规范、潜在 Bug、安全漏洞、性能优化和重构建议
3. **执行（输出）**：自动生成结构化审查报告并回写到 MR/PR 评论区，同时通过 Slack/企微/邮件推送通知，并将审查结论沉淀到知识库中实现持续学习

2.2 整体架构



2.3 创新点

1. **双模式触发设计**：既支持 Git 平台事件自动触发（生产级），又支持前端手动上传代码即时审查（演示级），兼顾实际生产与比赛演示场景
2. **知识库自进化**：审查结论不仅输出给用户，还会反向写入知识库（Notion/Airtable），形成"发现问题 → 更新规范 → 预防同类问题"的正反馈闭环
3. **多 AI 模型兼容**：通过 HTTP Request 节点设计，可灵活切换 GPT-4o、DeepSeek、Kimi、Claude 等模型，不绑定单一供应商
4. **与现有开发流程零侵入集成**：基于 Git webhook 触发，无需修改团队现有 CI/CD 流程，AI 审查作为"第二审查员"自动接入

三、技术选型与实现路径

3.1 技术选型理由

组件	选型	理由
工作流引擎	n8n	开源、节点丰富、可视化编排、支持 400+ 集成、有 SDK 可编程生成
AI 引擎	OpenAI GPT-4o（主）+ DeepSeek/Kimi（备选）	GPT-4o 支持 JSON 结构化输出，n8n 有原生集成；DeepSeek/Kimi 体现国产化方案
代码仓库	GitLab（主）+ GitHub（备）	GitLab 内置 Trigger 节点，支持 MR/Push/Note 多事件监听
知识库	Notion / Airtable	灵活的数据库能力，n8n 有原生节点支持，适合存储审查规范与结论
通知通道	Slack / Gmail / 企微	n8n 原生支持 Slack 和 Gmail；企微通过 Webhook 节点实现
前端演示	原生 HTML + Fetch	零依赖、可直接双击打开浏览器演示，适合比赛现场

3.2 实现路径

整个系统通过 **1 个核心 n8n 工作流** 实现，包含 **13 个节点**，分为两条并行处理路径：

路径 A — 手动演示模式（比赛演示 + 即时审查）：

```
Webhook (接收代码) → Code (标准化输入) → IF (手动分支)
→ Set (提取代码) → Merge → Build Prompt → AI Review → Parse Output
→ IF (输出路由) → Respond to Webhook (返回前端)
```

路径 B — GitLab 自动模式（生产部署）：

```
GitLab Trigger (MR 事件) → Code (标准化输入) → IF (GitLab 分支)
→ HTTP Request (拉取 Diff) → Code (过滤解析 diff) → Merge
→ Build Prompt → AI Review → Parse Output
→ IF (输出路由) → HTTP Request (回写 MR 评论)
```

3.3 关键节点实现

节点	功能	技术细节
Standardize Input	统一两种触发方式的数据格式	Code 节点（JavaScript）解析 webhook body / GitLab payload
Fetch GitLab Diff	获取 MR 代码变更	HTTP Request 调用 GitLab API `/projects/{id}/merge_requests/{iid}/changes`
GitLab: Parse Diff	过滤非代码文件、拼接 diff	Code 节点过滤 `.md/.txt/.png` 等非代码文件，合并变更片段
Build AI Prompt	构建结构化 Prompt	Set 节点注入 system prompt（角色设定 + 检查维度 + JSON 格式要求）
AI Review (OpenAI)	调用大模型执行审查	HTTP Request POST 到 OpenAI API，启用 `response_format: json_object`
Parse AI Output	解析 AI 返回为统一结构	Code 节点去除 markdown 代码块、JSON 解析、兜底容错
Respond to Webhook	返回审查结果给前端	Respond to Webhook 节点，返回完整 JSON 审查报告
Post GitLab Comment	回写 MR 评论	HTTP Request POST 到 GitLab `/projects/{id}/merge_requests/{iid}/notes`

3.4 AI Prompt 设计

```
你是一位资深代码审查专家。请对以下代码进行全面审查并输出结构化报告。
要求：
1. 确认编程语言
2. 检查代码规范（命名、格式、注释、可读性）
3. 检查潜在 Bug 和边界条件（空值、异常处理、资源泄漏、除零等）
4. 检查安全漏洞（SQL 注入、XSS、命令注入、硬编码密钥等）
5. 提供性能优化和重构建议

每个问题必须包含：
- 等级：严重 / 警告 / 建议
- 描述：具体问题
- 修复示例：可直接替换的代码片段

输出格式为 JSON：
{
  "detectedLanguage": "确认后的语言",
  "summary": "整体评价（1-2句话）",
  "issueCount": 3,
  "issues": [{"level": "...", "title": "...", "description": "...", "suggestion":
"..."}],
  "markdownReport": "用于展示的 Markdown 报告"
}
```

3.5 手动演示前端

使用 `demo/code-review-demo.html` 单文件实现：

- 支持 **文件拖拽/点击上传**（.js, .ts, .py, .java, .go, .cpp 等常见语言）
- 支持 **代码直接粘贴**
- **自动语言识别**：通过文件扩展名识别 + 代码特征关键字兜底识别
- **实时结果展示**：调用 n8n Webhook 后展示格式化审查报告

四、功能模块描述与交互流程

4.1 功能模块总览

模块	功能	对应 n8n 节点
触发模块	监听 Git 事件 / 接收手动提交	GitLab Trigger / Webhook
代码处理模块	解析 diff / 提取代码 / 语言识别	Code + Set
AI 审查模块	多维度代码分析	HTTP Request (OpenAI API)
报告生成模块	结构化报告 / Markdown 输出	Code
反馈模块	MR 评论回复 / 通知推送	HTTP Request + Slack / Gmail
知识库模块（规划）	审查结论沉淀 / 规范更新	Notion / Airtable

4.2 交互流程图 — 手动审查场景

```
用户打开 demo/code-review-demo.html
          │
          ▼
  上传代码文件 或 粘贴代码
          │
          ▼
  前端自动识别编程语言
          │
          ▼
  点击「开始智能审查」
          │
          ▼
  POST → n8n Webhook (/code-review-demo)
          │
          ▼
  n8n 工作流执行（13 节点）
          │
          ├── Standardize Input (识别来源为 manual)
          ├── IF 判断 → True 分支 (手动模式)
          ├── Manual: Extract Code
          ├── Build AI Prompt (构建 Prompt)
          ├── AI Review (调用 GPT-4o)
          ├── Parse AI Output (解析结果)
          └── IF 判断 → True → Respond to Webhook
          │
          ▼
  前端收到 JSON 审查报告
          │
          ▼
  展示 Markdown 格式报告（问题等级、描述、修复示例）
```

4.3 交互流程图 — GitLab 自动审查场景

```
开发者在 GitLab 创建 Merge Request
          │
          ▼
  GitLab Trigger 自动触发 n8n 工作流
          │
          ▼
  n8n 工作流执行
          │
          ├── Standardize Input (识别来源为 gitlab)
          ├── IF 判断 → False 分支 (GitLab 模式)
          ├── Fetch GitLab Diff (拉取代码变更)
          ├── GitLab: Parse Diff (过滤非代码文件、拼接 diff)
          ├── Build AI Prompt (构建 Prompt + 注入团队规范)
          ├── AI Review (调用 GPT-4o / DeepSeek)
          ├── Parse AI Output (解析结果)
          └── IF 判断 → False → Post GitLab Comment
          │
          ▼
  审查结果自动写入 MR 评论区
          │
          ▼
  团队成员收到 Slack / 邮件通知
```

4.4 知识库学习流程（规划）

```
每次审查完成
      │
      ▼
  审查结论写入知识库 (Notion Database)
      │
      ├── 新增问题类型 → 更新检查维度
      ├── 团队规范变更 → 注入下一次 Prompt
      └── 统计信息更新 → 团队代码质量画像
      │
      ▼
  定时工作流 C（每周）
      │
      ├── 查询本周审查数据
      ├── AI 生成质量趋势周报
      └── 发送通知
```

五、可扩展性与落地应用前景

5.1 跨场景复用

扩展方向	实现方式
其他代码平台	替换 Trigger 节点为 GitHub/BitBucket/Gitee
其他 AI 模型	通过 HTTP Request 节点切换 DeepSeek/Kimi/Claude/Ollama 等
其他通知通道	替换为企微/钉钉/飞书 Webhook 机器人
其他业务代码审查	将知识库中的团队规范替换为对应项目规范即可

5.2 部署成本与安全性

- **部署成本**：n8n 为开源项目，可自建（Docker 一键部署）；AI 模型按 API 调用量计费，单次审查成本约 0.01-0.05 元
- **数据安全**：代码片段仅通过 HTTPS 传输至 AI API，不落地外部存储；可对接本地部署的 Ollama 模型实现完全内网部署
- **稳定性**：n8n 支持工作流版本控制、重试机制、错误处理节点，确保审查流程高可用

5.3 与现有系统集成

现有系统	集成方式
GitLab CI/CD	在 CI pipeline 中添加 n8n Webhook 调用步骤
Jenkins	通过 Jenkins HTTP Request 插件调用 n8n Webhook
企业微信/钉钉	通过 Webhook 机器人接收审查通知
Jira/Linear	通过 n8n 内置节点创建 Bug/Issue 工单

5.4 后续优化路径

1. **多智能体协作审查**：引入多个 AI Agent（规范 Agent、安全 Agent、性能 Agent），各自负责一个维度，最终由 Merge 节点汇总
2. **向量数据库检索**：将历史审查结论存入向量数据库（Pinecone/Qdrant），审查时先检索相似案例，注入 Prompt 作为参考
3. **代码质量趋势可视化**：接入 Grafana 或 n8n 内置 Insights，展示代码质量变化趋势、问题类型分布、修复效率等指标
4. **交互式审查**：支持开发者针对 AI 审查结论进行追问和对话，AI 解释具体问题并生成修复代码
5. **增量学习优化**：定期收集开发者对 AI 审查结论的反馈（采纳/忽略/误报），用于优化 Prompt 和检查维度

5.5 落地价值

- **提升代码质量**：AI 审查可发现 80%+ 的常见问题，减轻人工审查压力
- **统一团队规范**：知识库驱动的规范检查，确保所有代码遵循统一标准
- **降低新人门槛**：新人提交的代码获得即时反馈，快速学习团队规范
- **加速交付节奏**：MR/PR 不再等待人工审查，AI 秒级响应，缩短迭代周期